

HERALD COLLEGE KATHMANDU

Week 2

Writing Dockerfiles

Containerising a MERN application · Tutorial 2 hrs · Workshop 2.5 hrs · Phase 1

In Week 1 you ran images other people built. This week you write the instructions that build one. That instruction file is called a **Dockerfile**. You will write two of them — one for the React frontend, one for the Express backend of a real MERN application — and build each into an image you can run.

By the end of this week you can

- explain what a Dockerfile is and how it builds an image *layer by layer*;
- write a Dockerfile for a Node.js application from an empty file;
- order instructions so that dependency installs stay *cached* between builds;
- build an image with `docker build` and run it with a published port;
- connect containers together — backend to database, and browser to backend — on a Docker network;
- diagnose the common Dockerfile and connection mistakes — broken caching, missing files, wrong ports, unreachable database.

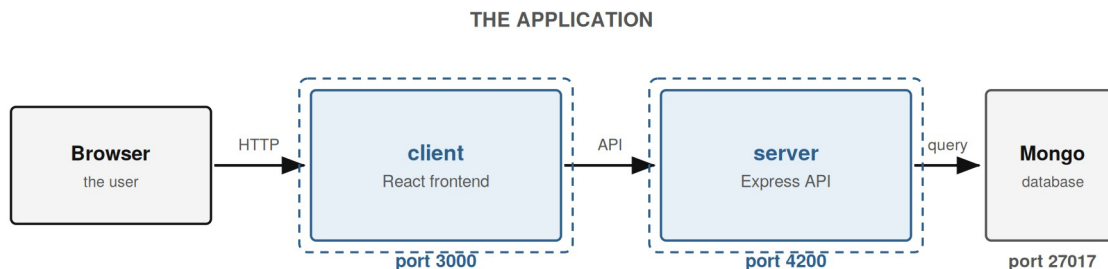
The project we will use

We containerise an existing open-source MERN CRUD application: `github.com/nurislam03/MERN-CRUD-Application`. It is deliberately simple and has exactly the structure we want — a `client` folder (React) and a `server` folder (Express), each an independent Node application. Clone it before the workshop:

```
git clone https://github.com/nurislam03/MERN-CRUD-Application.git
cd MERN-CRUD-Application
```

1 What We Are Building

A MERN application is three moving parts: a **React** frontend the user sees, an **Express** backend that serves data, and a **MongoDB** database that stores it. The frontend talks to the backend over HTTP; the backend talks to the database.



Two folders you will write a Dockerfile for:

client/Dockerfile and **server/Dockerfile**

MongoDB is not built by you — you run the official mongo image (Week 1).

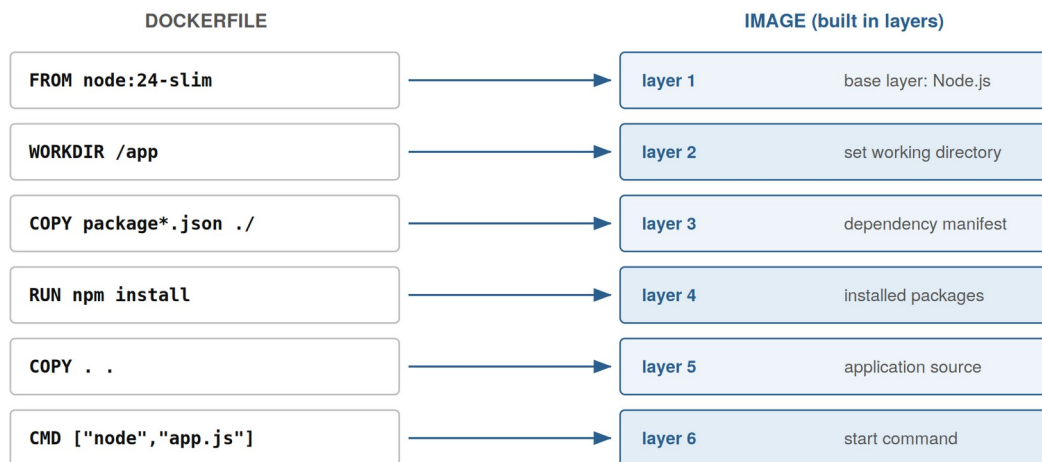
Figure 1. The application. You will write a Dockerfile for client and for server; MongoDB runs from its official image.

This matters for Docker: each part runs separately, so each gets its own image. We do not build one giant container for the whole app — we build one per service. This week focuses on the two you own, the client and the server. The database you already know how to run from Week 1, using the official mongo image — and in Sections 7 and 8 you will connect all three together so they behave as one application.

2 What a Dockerfile Is

A Dockerfile is a plain text file, named exactly `Dockerfile` with no extension, containing an ordered list of instructions. Each instruction describes one step in assembling your image. When you run `docker build`, Docker reads the file top to bottom and executes each step.

Crucially, **each instruction produces a layer** — a saved, cached snapshot stacked on the one before it. This layering is the single most important idea in this document, because it governs both how images are built and how fast they rebuild.



Each instruction, top to bottom...

...adds one stacked, cached layer.

Figure 2. Docker executes each instruction in order; each one adds a cached layer to the image.

The core instructions, which are all you need this week:

Instruction	What it does
FROM	Choose the base image to build on top of (here, Node.js)
WORKDIR	Set the working directory inside the image for the steps that follow
COPY	Copy files from your machine into the image
RUN	Execute a command at build time (e.g. install dependencies)
EXPOSE	Document which port the application listens on
CMD	The command that runs when a container starts from the image

3 The Backend Dockerfile, Step by Step

We start with the server because it is the simpler of the two. Open the `server` folder. We build the Dockerfile one instruction at a time, and explain each before adding the next.

Step 1 · Choose a base image

Every Dockerfile begins with `FROM`. We do not start from an empty machine — we start from an official image that already has Node.js installed.

```
# Start from Node.js 24 (the current LTS), slim variant
FROM node:24-slim
```

Why `24-slim` and not `latest`? Two reasons you met in Week 1: pinning the version (24) keeps builds reproducible, and the `slim` variant is a much smaller image than the default.

Step 2 · Set the working directory

```
WORKDIR /app
```

This creates `/app` inside the image and moves into it. Every command after this runs from `/app`, so we do not have to write full paths repeatedly. Without it, files would land in the image root, mixed in with system files.

Step 3 · Copy the dependency manifest, then install

The order of the next two steps is deliberate and is the most important lesson of the week. We copy *only* the package files first, install dependencies, and copy the rest of the code afterwards.

```
# Copy only the dependency list first
COPY package*.json ./

# Install dependencies
RUN npm install
```

Section 4 explains exactly why this order matters. For now, take it on trust that copying the manifest before the source code saves you large amounts of time on every rebuild.

Step 4 · Copy the application code

```
# Now copy everything else
COPY . .
```

With dependencies installed, we copy the remaining source. The first `.` is your project folder on your machine; the second is `/app` inside the image.

Step 5 · Document the port and set the start command

```
# The Express server listens on 4200
EXPOSE 4200

# Command that runs when the container starts
CMD ["node", "app.js"]
```

`EXPOSE` is documentation — it records the port for anyone reading the file but does not publish it (you still do that with `-p` at run time, as in Week 1). `CMD` is the command Docker runs to start your app.

[The complete server/Dockerfile](#)

```
FROM node:24-slim
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 4200
CMD ["node", "app.js"]
```

4 Why Instruction Order Matters

Return to the claim from Step 3: copy the dependency manifest before the source code. Here is why, and it comes straight from how layers cache.

When you rebuild an image, Docker reuses the cached layer for any instruction whose inputs have not changed. The moment one layer changes, **every layer below it must be rebuilt** — the cache is invalidated from that point down.



Figure 3. Copying source before installing forces a full reinstall on every code change. Copying the manifest first keeps the install cached.

Your dependencies change rarely; your source code changes constantly. If `COPY . .` comes before `npm install`, then editing a single line of code changes the copy layer, which invalidates the install layer, so `npm install` runs again — every single build. By copying `package*.json` first, the install layer only rebuilds when your dependencies actually change. This is the difference between a two-second rebuild and a two-minute one.

5 Building and Running the Backend

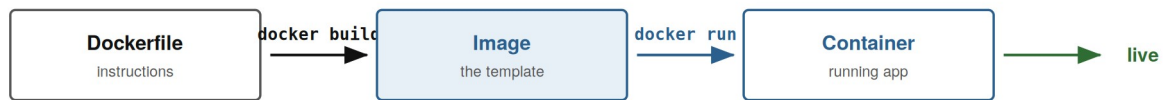
With the Dockerfile written, build an image from it. The `-t` flag tags it with a name; the `.` tells Docker to build using the Dockerfile in the current folder.

```
docker build -t mern-server .
```

Watch the output: Docker runs each instruction as a numbered step. Build a second time without changing anything — notice it now says `CACHED` against most steps and finishes almost instantly. That is the layer cache at work.

Now run a container from your image, publishing the port:

```
docker run -d -p 4200:4200 --name server mern-server
docker logs server
```



Week 1 gave you the last two arrows. This week you add the first one — building your own image.

Figure 4. build turns a Dockerfile into an image; run turns an image into a container.

Read the logs and you will see a problem: the server starts, then fails to connect to a database, because there is no database running yet. That is the next section — and it is where most of the real learning happens.

6 The Frontend Dockerfile

The `client` folder is a React application. Its Dockerfile follows the same shape as the server — the skeleton you learned transfers directly. Open the `client` folder and create a Dockerfile:

```
FROM node:24-slim
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
# React's dev server runs on port 3000
EXPOSE 3000
CMD ["npm", "start"]
```

Only two lines differ from the server: the port is 3000 (React's development server), and the start command is `npm start` rather than `node app.js`. Everything you learned about base images, working directories, and caching order applies unchanged.

Build and run it:

```
docker build -t mern-client .
docker run -d -p 3000:3000 --name client mern-client
```

Open `http://localhost:3000` and the React app appears. But it cannot load or save any data yet — the pieces are running, but nothing is connected. Sections 7 and 8 fix that.

A NOTE ON `.DOCKERIGNORE`

Alongside each Dockerfile, create a file named `.dockerignore` containing the single line `node_modules`. This stops Docker copying your local dependency folder into the image during `COPY . .`, which would be slow and would overwrite the clean install from `npm install`. It works just like `.gitignore`.

7 Connecting the Backend to MongoDB

Three containers running side by side are not an application. The backend needs to reach the database, and by default it cannot — because each container has its own private network, exactly as you saw with ports in Week 1.

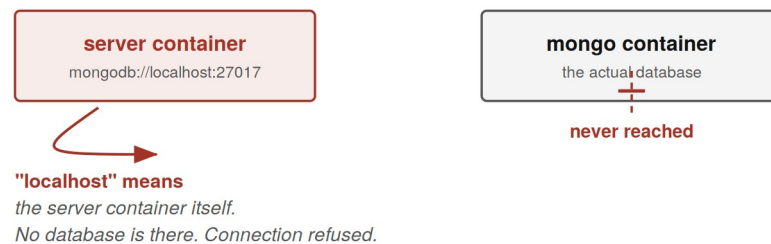
7.1 Why localhost does not work here

Open the project's `server/app.js` and you will find this line:

```
mongoose.connect('mongodb://localhost:27017/CrudDB')
```

On a single machine that is fine. Inside a container it is broken, and the reason is the single most important networking idea this week. To a program running inside the server container, **localhost means “this container, myself”** — not your computer, and certainly not some other container. So `localhost:27017` tells the server to look for a database *inside itself*. There is none there, and the connection is refused.

WRONG



RIGHT

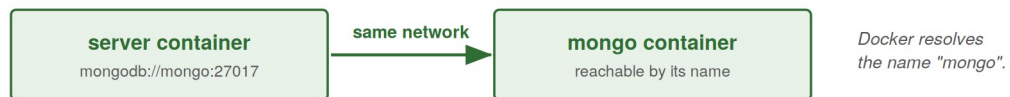


Figure 5. Inside a container, localhost points back at that same container — not at the database container.

7.2 The fix: a shared network and a name

Docker lets you create a network and attach containers to it. Containers on the same network can reach each other **by their container name**, which Docker resolves automatically. Put the server and mongo containers on one network, and the server can reach the database at `mongo:27017` instead of `localhost:27017`.

First, make the connection string configurable instead of hardcoded. Change that line in `server/app.js` to read the host from an environment variable, falling back to localhost for non-Docker use:

```
const mongoHost = process.env.MONGO_HOST || 'localhost';
mongoose.connect(`mongodb://${mongoHost}:27017/CrudDB`);
```

Rebuild the server image so the change is included: `docker build -t mern-server ..`

7.3 Create the network and start both containers

Create a network, start MongoDB on it, then start the server on the same network and point it at the database by name:

```
# 1. Create a network the containers will share
docker network create mern-net

# 2. Start MongoDB on that network (official image, Week 1)
```

```
docker run -d --network mern-net --name mongo mongo
```

```
# 3. Start the server on the same network,  
#    telling it to reach the database at 'mongo'  
docker run -d --network mern-net --name server \  
-e MONGO_HOST=mongo -p 4200:4200 mern-server
```

The `--name mongo` is what makes this work: it is the hostname the server now looks up. Check `docker logs server` — the database error is gone, and the server reports it has started.

WHY NOT PUBLISH MONGO'S PORT?

Notice the mongo container has no `-p`. Only the *server* needs to reach it, and the server is on the same network, so it uses the name directly. You publish a port only when something **outside** the network — like your browser — needs in. The database should not be exposed to the outside, so it stays unpublished.

8 Connecting the Frontend to the Backend

Now the frontend. It is tempting to assume the React app also reaches the server by its container name — but it does not, and understanding why cements the whole networking picture.

8.1 The frontend runs in the browser, not the container

Look at how the client calls the backend, in `client/src/components/ItemService.js`:

```
axios.get('http://localhost:4200/items')
```

Here **localhost is correct**, and the reason is subtle. The React container only *serves* the JavaScript to your browser; the code then actually **runs in the browser, on your machine** — the host. So when that code says `localhost:4200`, “localhost” means your computer, and it reaches the server through the port you published with `-p 4200:4200`. It does *not* go through the Docker network at all, which is why the container name `server` would not work here.

THE RULE WORTH MEMORISING

Container-to-container (backend → database): same Docker network, reach by **container name**.

Browser-to-container (frontend → backend): the code runs on the host, so reach by **localhost and a published port**.

8.2 The whole application, started in order

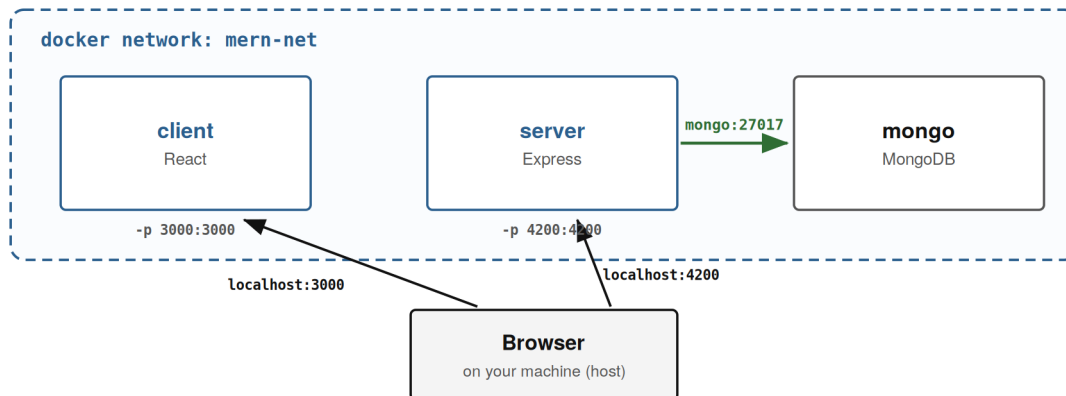
Putting it all together, here is every command to bring the full application up from nothing:

```
# network
docker network create mern-net

# database (no published port – internal only)
docker run -d --network mern-net --name mongo mongo

# backend (on the network, reaches mongo by name; port published for the browser)
docker run -d --network mern-net --name server \
  -e MONGO_HOST=mongo -p 4200:4200 mern-server

# frontend (port published for the browser)
docker run -d --network mern-net --name client \
  -p 3000:3000 mern-client
```



Inside the network, containers reach each other by name (green). From the browser, you reach published ports on localhost (black).

Figure 6. The connected system. Green: container-to-container by name, over the network. Black: browser-to-container by localhost, over published ports.

Open `http://localhost:3000`, add an item, and reload. It persists — the click travelled from your browser to the backend on port 4200, the backend saved it to MongoDB at `mongo:27017`, and reading it back returned your data. All three containers, connected.

9 Exercises

Each exercise gives you a problem to reproduce, then asks you to fix it. Use the material above and the Docker reference at docs.docker.com/reference/dockerfile/.

A The rebuild that is always slow

PROBLEM

Write the server Dockerfile with the copy order reversed — `COPY . .` placed *before* `RUN npm install`. Build it. Then change one line in `app.js` and build again.

The second build reinstalls every dependency, taking just as long as the first.

YOUR TASK

Explain why, then fix the ordering so the second build reuses the cached install.

Hint. Re-read Section 4 and Figure 3. Which layer changed when you edited the code, and what does that do to the layers below it?

B The image that will not start

PROBLEM

Build a server image whose final line reads `CMD ["node", "server.js"]`, but the project's entry file is actually `app.js`. Run it.

The container exits immediately. `docker ps` shows nothing running.

YOUR TASK

Find out what went wrong and correct the Dockerfile so the container stays up.

Hint. A container stops when its main process ends. Run `docker logs <id>` on the exited container — Node will tell you plainly what it could not find.

C The page that will not load

PROBLEM

Build and run the client image, but publish it with `-p 3000:5000`.

The container runs, but `http://localhost:3000` gives a connection error.

YOUR TASK

Explain the mistake and run it so the page loads.

Hint. This is the Week 1 port lesson meeting your new Dockerfile. Which port does the React container actually listen on — and which side of `-p` must match it?

D The enormous, slow build

PROBLEM

Delete the `.dockerignore` file from the client folder, then rebuild the image.

The build is noticeably slower and the image is much larger than before.

YOUR TASK

Explain what got copied in that should not have, and restore the fast build.

Hint. What sits in the client folder after you run `npm install` locally? Section 6's note names the file that prevents this.

E The database it cannot find

PROBLEM

Start mongo and server both, but leave the server's connection pointing at localhost (no MONGO_HOST), or forget to put them on the same network.

The server starts but its logs show it cannot connect to the database.

YOUR TASK

Explain, in terms of container networking, why the server cannot see the database — then connect them so it can.

Hint. Re-read Section 7 and Figure 5. What does localhost mean to a program inside the server container, and what has to be true for one container to reach another by name?

F The item that will not save

PROBLEM

Run all three containers, but publish the backend on -p 4300:4200 instead of 4200:4200. Open the app and try to add an item.

The page loads, but adding an item fails silently — nothing is saved.

YOUR TASK

Explain why the frontend cannot reach the backend, given that the backend is running fine.

Hint. The React code calls the backend at localhost:4200 from your browser (Section 8). If nothing on the host answers on 4200, the request goes nowhere. Which host port did you actually publish?

10 When It Goes Wrong

Symptom	Cause and fix
npm install runs on every build	COPY . sits before RUN npm install, so any code change invalidates the install layer. Copy package*.json first (Section 4).
Cannot find module ... / file not found	The CMD points at a file that does not exist in the image — wrong name, or not copied. Check the entry file name and that COPY . ran.
Container exits immediately	The start command failed. Read docker logs for the exact error; it is almost always a wrong CMD or a missing dependency.
Server cannot connect to MongoDB	The server is using localhost, or the two containers are not on the same network. Set MONGO_HOST=mongo and run both with --network mern-net (Section 7).
Frontend loads but data never saves	The browser calls the backend on localhost:4200; if the backend's published host port is not 4200, nothing answers. Publish it with -p 4200:4200 (Section 8).
Runs, but browser cannot reach it	The -p mapping's container-side port does not match EXPOSE / the app's real port. Match them (Week 1, Section 5).
Image is huge and builds slowly	node_modules is being copied in. Add a .dockerignore containing node_modules.

11 Before Next Week

Check you can do each of these without looking anything up.

1. Write a Node.js Dockerfile from memory, in the correct instruction order.
2. Explain why COPY package*.json ./ comes before COPY . ..
3. Explain why a container reaches another by name but the browser reaches a container by localhost.
4. Create a network, and connect the backend to MongoDB using MONGO_HOST.
5. Build both images, start all three services connected, and save an item in the app.

Looking ahead: you now have three containers started by hand, one command at a time. Next week you will define all of them in a single file and start the whole application with one command, using Docker Compose.

Reference

- Dockerfile reference — docs.docker.com/reference/dockerfile/
- Build best practices — docs.docker.com/build/building/best-practices/
- Node.js image — hub.docker.com/_/node
- The project — github.com/nurislam03/MERN-CRUD-Application